Основы CSS - Изучение веб-разработки | MDN

Назад

Обзор: Getting started with the web

Далее

CSS (Cascading Style Sheets) — это код, который вы используете для стилизации вашей веб-страницы. *Основы CSS* помогут вам понять, что вам нужно для начала работы. Мы ответим на такие вопросы как: Как сделать мой текст черным или красным? Как сделать так, чтобы контент появлялся в определённом месте на экране? Как украсить мою веб-страницу с помощью фоновых изображений и цветов?

Так что же такое CSS?

Как и HTML, CSS на самом деле не является языком программирования. Это не язык разметки - это *язык таблицы стилей*. Это означает, что он позволяет применять стили выборочно к элементам в документах HTML. Например, чтобы выбрать **все** элементы абзаца на HTML странице и изменить текст внутри них с чёрного на красный, вы должны написать этот CSS:

```
p {
  color: red;
}
```

Давайте попробуем: вставьте эти три строки CSS в новый файл в ваш текстовый редактор, а затем сохраните файл как `style.css` в вашей папке `styles`.

Но нам всё равно нужно применить CSS к нашему HTML документу. В противном случае, CSS стиль не повлияет на то, как ваш браузер отобразит HTML документ. (Если вы не следили за нашим проектом, то прочитайте раздел Работа с файлами и Основы HTML, чтобы узнать, что вам нужно сделать в первую очередь.)

Откройте ваш файл `index.html` и вставьте следующую строку куда-нибудь в шапку, между `<head>` и `</head>` тегами:

```
<link href="styles/style.css" rel="stylesheet" type="text/css" />
```

Сохраните `index.html` и загрузите его в вашем браузере. Вы должны увидеть что-то вроде этого:

working together to keep the Internet alive and accessible, so people worldwide can be informed contributors and creators of the Web. We believe this act of human collaboration across an open platform is essential to individual growth and our collective future.

Read the Mozilla Manifesto to learn even more about the values and principles that guide the pursuit of our mission.

Если текст вашего абзаца теперь красный, примите наши поздравления! Вы написали свой первый успешный CSS!

Анатомия набора правил CSS

Давайте взглянем на вышеупомянутый CSS немного более подробно:

Вся структура называется **набором правил** (но зачастую для краткости "правило"). Отметим также имена отдельных частей:

Селектор (Selector)
: Имя HTML-элемента в начале набора правил. Он выбирает элемент(ы) для применения стиля (в данном случае, элементы `p`). Для стилизации другого элемента, просто измените селектор.

Объявление (Declaration)
: Единственное правило, например `color: red;` указывает, какие из **свойств** элемента вы хотите стилизовать.

Свойства (Properties)
: Способы, которыми вы можете стилизовать определённый HTML-элемент (в данном случае, `color` является свойством для элементов `<p>`). В CSS вы выбираете, какие свойства вы хотите затронуть в вашем правиле.

Значение свойства (Property value)
: Справа от свойства, после двоеточия, у нас есть **значение свойства**, которое выбирает одно из множества возможных признаков для данного свойства (существует множество значений `color`, помимо `red`).

Обратите внимание на важные части синтаксиса:

Каждый набор правил (кроме селектора) должен быть обёрнут в фигурные скобки (`{}`).

В каждом объявлении необходимо использовать двоеточие (`:`), чтобы отделить свойство от его значений.

В каждом наборе правил вы должны использовать точку с запятой (`;`), чтобы отделить каждое объявление от следующего.

Таким образом, чтобы изменить несколько значений свойств сразу, вам просто нужно написать их, разделяя точкой с запятой,

например так:

```
p {
  color: red;
  width: 500px;
  border: 1px solid black;
}
```

Выбор нескольких элементов

Вы также можете выбрать несколько элементов разного типа и применить единый набор правил для всех из них. Добавьте несколько селекторов, разделённых запятыми. Например:

```
p,
li,
h1 {
  color: red;
}
```

Разные типы селекторов

Существует множество различных типов селектора. Выше мы рассматривали только **селектор элементов**, который выбирает все элементы определённого типа в HTML документе. Но мы можем сделать выбор более конкретным. Вот некоторые из наиболее распространённых типов селекторов:

Имя селектора	Что выбирает	Пример
Селектор элемента (иногда называемый селектором тега или типа)	Все HTML-элемент(ы) указанного типа.	`p` Выбирает `<p>`
ID селектор	Элемент на странице с указанным ID на данной HTML. Лучше всего	`#my-id` Выбирает `<p id="my-id">`

Имя селектора	Что выбирает	Пример
	использовать один элемент для каждого ID (и конечно один ID для каждого элемента), даже если вам разрешено использовать один и тот же ID для нескольких элементов.	или `<a id="my-id">`
Селектор класса	Элемент(ы) на странице с указанным классом (множество экземпляров класса может объявляться на странице).	`.my-class` Выбирает `<p class="my-class">` и `<a class="my-class">`
Селектор атрибута	Элемент(ы) на странице с указанным атрибутом.	`img[src]` Выбирает `<img src="myimage.png">` но не `<img>`
Селектор псевдокласса	Указанные элемент(ы), но только в случае определённого состояния, например, при наведении курсора.	`a:hover` Выбирает `<a>`, но только тогда, когда указатель мыши наведён на ссылку.

Существует ещё много селекторов для изучения, и вы можете найти более подробный список в нашем Руководстве селекторов (en-US).

Шрифты и текст

Теперь, когда мы изучили некоторые основы CSS, давайте добавим ещё несколько правил и информацию в наш файл `style.css`, чтобы наш пример хорошо выглядел. Прежде всего, давайте сделаем, чтобы наши шрифты и текст выглядели немного лучше.

Прежде всего, вернитесь и найдите вывод из Google Fonts, который вы уже где-то сохранили. Добавьте элемент `<link>` где-нибудь внутри шапки вашего `index.html` (снова, в любом месте между тегами `<head>` и `</head>`). Это будет выглядеть примерно так:

```
<link
  href="http://fonts.googleapis.com/css?family=Open+Sans"
  rel="stylesheet"
  type="text/css" />
```

Этот код связывает вашу страницу с таблицой стилями, которая загружает семейство шрифтов Open Sans вместе с вашей страницей и позволяет вам применять их к вашим HTML-элементам используя свою собственную таблицу стилей.

Затем, удалите существующее правило в вашем `style.css` файле. Это был хороший тест, но красный текст, на самом деле, не очень хорошо выглядит.

Добавьте следующие строки в нужное место, заменив строку `placeholder` актуальной `font-family` строкой, которую вы получили из Google Fonts. (`font-family` просто означает, какой шрифт(ы) вы хотите использовать для вашего текста). Это правило устанавливает глобальный базовый шрифт и размер шрифта для всей страницы (поскольку `<html>` является родительским элементом для всей страницы, и все элементы внутри него наследуют такой же `font-size` и `font-family`):

```
html {
  font-size: 10px; /* px значит 'пиксели': базовый шрифт будет 10 пикселей в высоту  */
  font-family: placeholder: здесь должно быть имя шрифта из Google fonts
}
```

Примечание: Все в CSS документе между `/*` и `*/` является **CSS комментарием**, который браузер игнорирует при исполнении кода. Это место, где вы можете написать полезные заметки о том, что вы делаете.

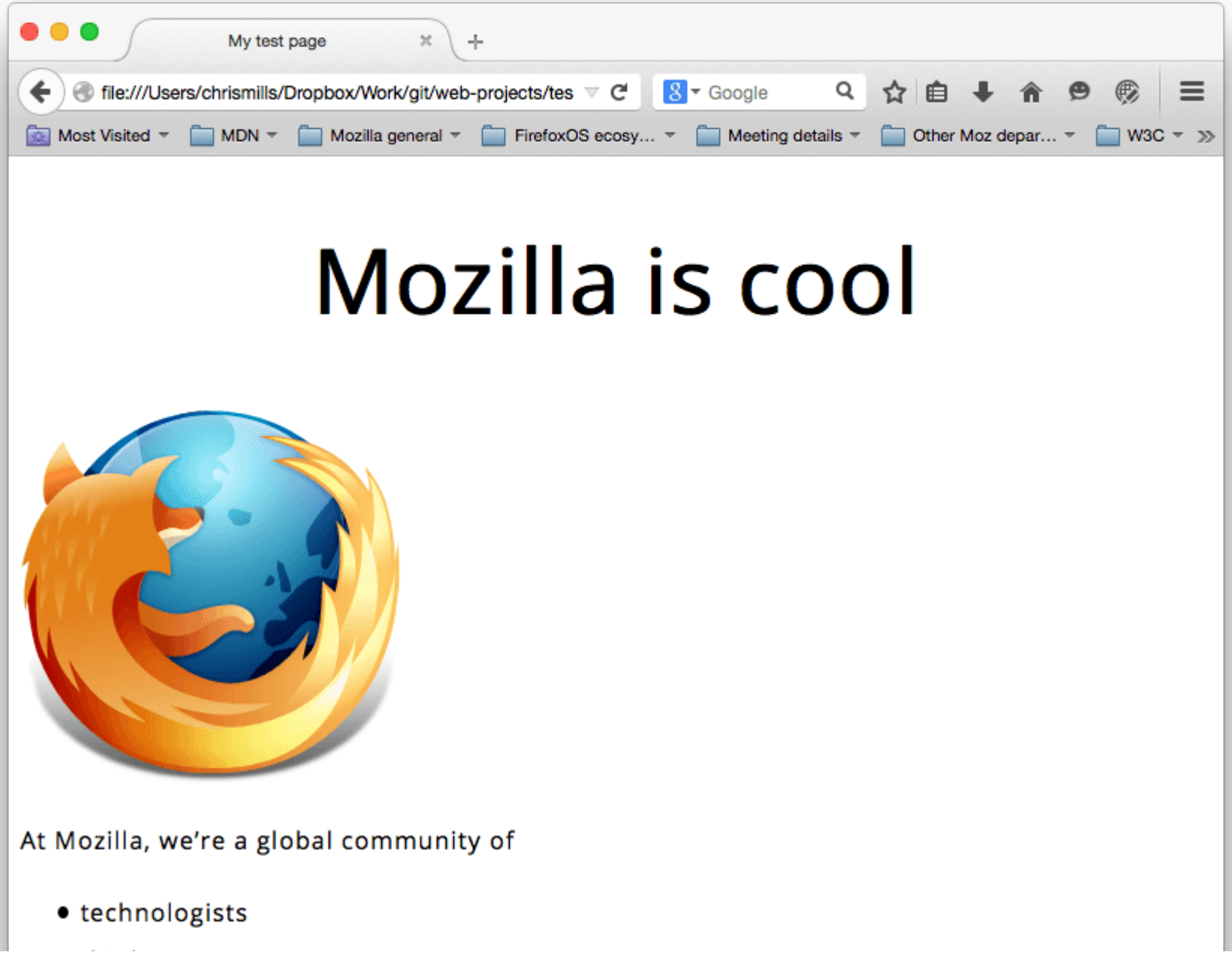
Теперь мы установим размер шрифта для элементов, содержащих текст внутри HTML тела (`<h1>` (en-US), `<li>`, и `<p>`). Мы также отцентрируем текст нашего заголовка и установим некоторую высоту строки и расстояние между буквами в теле документа, чтобы сделать его немного более удобным для чтения:

```
h1 {
  font-size: 60px;
  text-align: center;
}

p,
li {
  font-size: 16px;
  line-height: 2;
  letter-spacing: 1px;
}
```

Вы можете настроить значения `px` так, как вам нравится, чтобы ваш дизайн выглядел так, как вы хотите, но, в общем, ваш дизайн

должен выглядеть вот так:



- thinkers
- builders

working together to keep the Internet alive and accessible, so people worldwide can be informed contributors and creators of the Web. We believe this act of human collaboration across an open platform is essential to individual growth and our collective future.

Read the Mozilla Manifesto to learn even more about the values and principles that guide the pursuit of our mission.

Блоки, блоки и ещё раз блоки

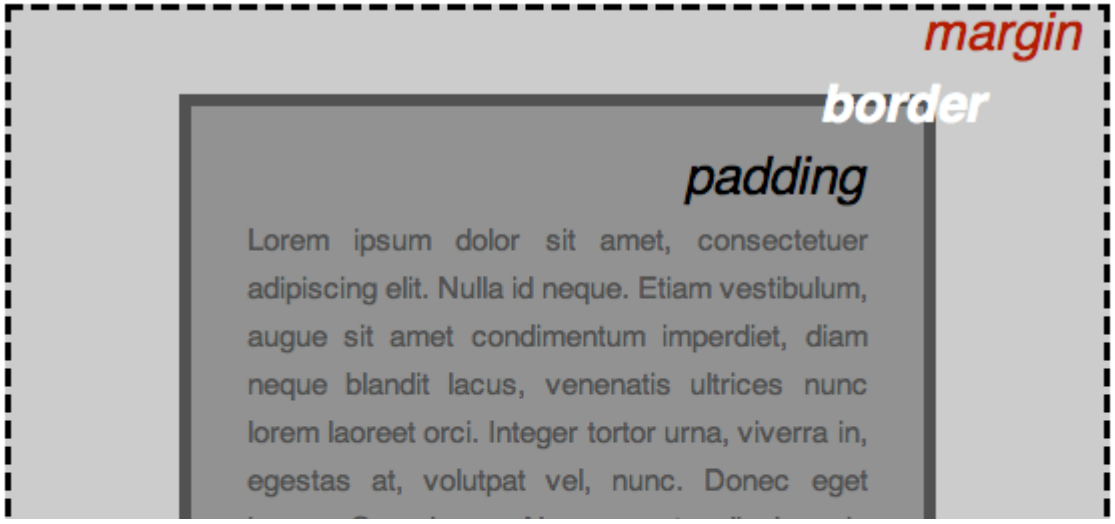
Одна вещь, которую вы заметите в написании CSS, заключается в том, что многое из этого касается блоков - настройка их размера, цвета, положения и т.д. Большинство HTML-элементов на странице можно рассматривать как блоки, расположенные друг над другом.

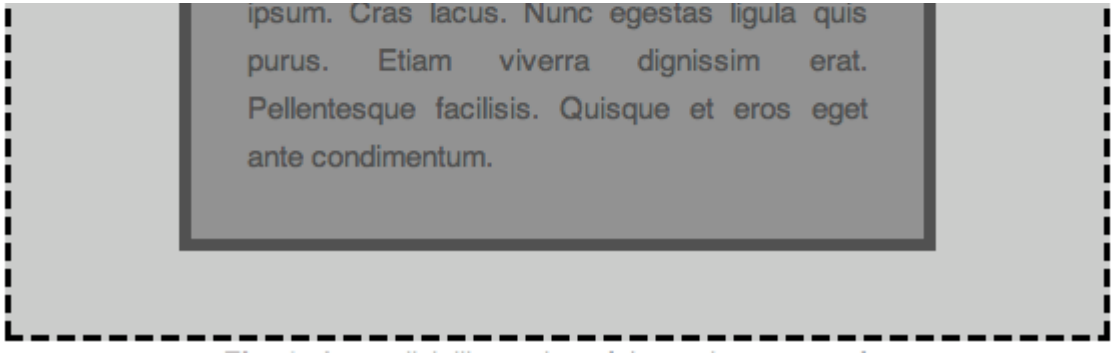
Не удивительно, макет CSS основан, главным образом, на *блочной модели (box model)*. Каждый из блоков, занимающий пространство на вашей странице имеет такие свойства, как:

`padding`, пространство только вокруг контента (например, вокруг абзаца текста)

`border`, сплошная линия, которая расположена рядом с padding

`margin`, пространство вокруг внешней стороны элемента





В этом разделе мы также используем:

`width` (ширину элемента)

`background-color`, цвет позади контента и padding элементов

`color`, цвет контента элемента (обычно текста)

`text-shadow`: устанавливает тень на тексте внутри элемента

`display`: устанавливает режим отображения элемента (пока что не волнуйтесь об этом)

Итак, давайте начнём и добавим больше CSS на нашей странице! Продолжайте добавлять эти новые правила, расположенные в нижней части страницы, и не бойтесь экспериментировать с изменением значений, чтобы увидеть, как это работает.

Изменение цвета страницы

```
html {
  background-color: #00539f;
}
```

Это правило устанавливает цвет фона для всей страницы. Измените код цвета сверху, на цвет который вы выбрали при планировании вашего сайта.

Разбираемся с телом

```
body {
  width: 600px;
  margin: 0 auto;
  background-color: #ff9500;
  padding: 0 20px 20px 20px;
  border: 5px solid black;
}
```

Теперь для `<body>` элемента. Здесь есть немало деклараций, так что давайте пройдём через них всех по одному:

`width: 600px;` — заставляет тело быть всегда 600 пикселей в ширину.

`margin: 0 auto;` — когда вы устанавливаете два значения для таких свойств как `margin` или `padding`, первое значение элемента влияет на верхнюю **и** нижнюю сторону (делает их `0` в данном случае), и второе значение на левую **и** правую сторону (здесь, `auto` является особым значением, которое делит доступное пространство по горизонтали поровну слева и справа). Вы также можете использовать один, три или четыре значения, как описано здесь.

`background-color: #FF9500;` — как и прежде, устанавливает цвет фона элемента. Я использовал красновато-оранжевый для тела, в отличие от темно-синего цвета для `<html>` элемента, но не стесняйтесь и экспериментируйте.

`padding: 0 20px 20px 20px;` — у нас есть четыре значения, установленные для padding, чтобы сделать немного пространства вокруг нашего контента. В этот раз мы не устанавливаем padding на верхней части тела, но делаем 20 пикселей слева, снизу и справа. Значения устанавливаются сверху, справа, снизу, слева, в таком порядке.

`border: 5px solid black;` — просто устанавливает сплошную чёрную рамку шириной 5 пикселей со всех сторон тела.

Позиционирование и стилизация нашего заголовка главной страницы

```
h1 {
  margin: 0;
  padding: 20px 0;
```

```
  color: #00539f;
  text-shadow: 3px 3px 1px black;
}
```

Вы, возможно, заметили, что есть ужасный разрыв в верхней части тела. Это происходит, потому что браузеры применяют некоторый **стиль по умолчанию** для элемента `<h1>` (en-US) (по сравнению с другими), даже если вы не применяли какой-либо CSS вообще! Это может звучать как плохая идея, но мы хотим, чтобы веб-страница без стилей имела базовую читаемость. Чтобы избавиться от разрыва, мы переопределили стиль по умолчанию, установив `margin: 0;`.

Затем мы установили заголовку верхний и нижний padding на 20 пикселей, и сделали текст заголовка того же цвета, как и цвет фона html.

Здесь, мы использовали одно довольно интересное свойство - это `text-shadow`, которое применяет тень к текстовому контенту элемента. Оно имеет следующие четыре значения:

Первое значение пикселей задаёт **горизонтальное смещение** тени от текста — как далеко она движется поперёк: отрицательное значение должно двигать её влево.

Второе значение пикселей задаёт **вертикальное смещение** тени от текста — как далеко она движется вниз, в этом примере: отрицательное значение должно переместить её вверх.

Третье значение пикселей задаёт **радиус размытия** тени — большее значение будет означать более размытую тень.

Четвёртое значение задаёт основной цвет тени.

И вновь попробуйте поэкспериментировать с различными значениями, чтобы посмотреть, что вы можете придумать.

Центрирование изображения

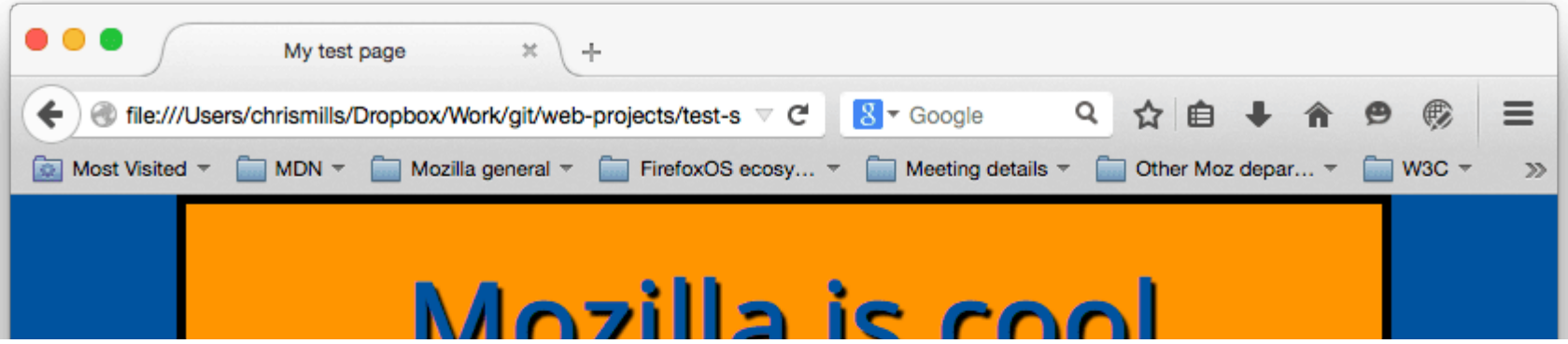
```
img {
  display: block;
  margin: 0 auto;
```

```
}
```

В заключение, мы отцентрируем изображение, чтобы оно лучше выглядело. Мы можем использовать `margin: 0 auto` уловку снова, как мы это делали раньше для body, но мы также должны сделать кое-что ещё. Элемент `<body>` является **блочным**, это значит, что он занимает всю ширину страницы и может иметь margin и другие значения отступов, применяемых к нему. Изображения, наоборот, являются **строчными** элементами, то есть они этого не могут. Таким образом, чтобы применить margin к изображению, мы должны дать изображению блочное поведение с помощью `display: block;`.

Примечание: Приведённые выше инструкции предполагают, что вы используете изображение меньшей ширины, чем заданная для `<body>` (600px). Если ваше изображение больше, тогда оно выйдет за границы `<body>` и займёт пространство страницы. Чтобы исправить это, вы можете: 1) уменьши ширину изображения используя графический редактор, или 2) изменить размер изображения используя CSS путём установки свойства `width` для `<img>` меньше, чем у `<body>`.

Примечание: Не стоит беспокоиться, если вы ещё не понимаете `display: block;` и различия между блочным/строчным. Вы поймёте, когда будете изучать CSS более подробно. Вы можете узнать больше о различных доступных значениях display на нашей странице о свойстве display.

Заключение

Если вы следовали всем инструкциям в этой статье, вы должны получить страницу, которая выглядит примерно так (вы также можете посмотреть нашу версию здесь):